

MODULE - 3

1. MERGE SORT (EASY) :-

Problem : Sort the given package value in ascending order using Merge Sort algorithm.

Constraints : $1 \leq N \leq 100000$
 $100 \leq \text{package} \leq 10000$

Algorithm :

1. Read the no. of elements (N).
2. Read the array elements.
3. Divide the array recursively into two halves until each subarray contains one element.
4. Merge the subarrays by comparing their elements in ascending order.
5. Print the sorted array.

Java Code :

```
import java.util.Scanner;  
public class MergeSortEasy {
```

```
    static void merge(int arr[], int left, int mid,  
                      int right) {  
        int n1 = mid - left + 1;  
        int n2 = right - mid;
```

```
        int L[] = new int[n1];  
        int R[] = new int[n2];
```

```
for (int i = 0; i < n1; i++)  
    L[i] = arr[left + i];
```

```
for (int j = 0; j < n2; j++)  
    R[j] = arr[mid + 1 + j];
```

```
int i = 0; j = 0; k = left;
```

```
while (i < n1 && j < n2) {  
    if (L[i] <= R[j]) {  
        arr[k] = L[i];  
        i++;
```

```
    } else {  
        arr[k] = R[j];  
        j++;
```

```
    }  
    k++;
```

```
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;
```

```
}
```

```
}
```

```
static void mergeSort(int arr[], int left,  
                       int right) {
```

```
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```

```

        mergesort(arr, mid+1, right);
        merge(arr, left, mid, right);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();

    mergeSort(arr, 0, n-1);

    for (int i = 0; i < n; i++)
        System.out.println(arr[i] + " ");

    sc.close();
}
}

```

Input : 6
450 1200 800 650 300 1500

```
● PS C:\Users\Dell> cd "c:\Users\Dell\Downloads\" ; if ($?) { javac MergeSortEasy.java } ; if ($?) { java MergeSortEasy }  
6  
450 1200 800 650 300 1500  
300 450 650 800 1200 1500  
○ PS C:\Users\Dell\Downloads> █
```

2. MergeSort (MEDIUM):-

Problem: Sort the given processing times in ascending order using Merge Sort.

After sorting, find the median processing time, count the no. of orders greater than median, calculate the difference b/w the max and min time.

Constraints: $1 \leq N \leq 100000$
 $1 \leq \text{Time} \leq 10000$

Algo: 1). Read N.

2. Read the time process into an array.

3. Sort the array using merge sort.

4. Print the sorted array.

5. Find the Median.

- N is odd, median = middle.

- N is even, median = average of two middle one.

6. Count the no. of element greater than median.

7. Diff. b/w max and min element.

8. Display all results.

Java Code: `import java.util.Scanner;`

`public class MergeSortMedium {`

`static void merge (int arr[], int left, mid, int right) {`

```
int n1 = mid - left + 1;  
int n2 = right - mid;
```

```
int L[] = new int[n1];  
int R[] = new int[n2];
```

```
for (int i = 0; i < n1; i++)  
    L[i] = arr[left + i];
```

```
for (int j = 0; j < n2; j++)  
    R[j] = arr[mid + 1 + j];  
int i = 0, j = 0, k = left;
```

```
while (i < n1 && j < n2) {  
    if (L[i] < R[j]) {  
        arr[k] = L[i];  
        i++;
```

```
    } else {  
        arr[k] = R[j];  
        j++;
```

```
        k++;  
    }
```

```
    while (i < n1) {  
        arr[k] = L[i];  
        i++;  
        k++;  
    }
```

```
    while (j < n2) {  
        arr[k] = R[j];  
        j++;  
        k++;  
    }
```



```
static void mergeSort (int arr[], int left, int  
right) {
```

```
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;  
        mergeSort (arr, left, mid);  
        mergeSort (arr, mid + 1, right);  
        merge (arr, left, mid, right);  
    }
```

```
public static void main (String[] args) {
```

```
    Scanner sc = new Scanner (System.in);  
    int n = sc.nextInt();  
    int arr[] = new int[n];
```

```
    for (int i = 0; i < n; i++)  
        arr[i] = sc.nextInt();  
    MergeSort (arr, 0, n-1);
```

```
    System.out.println ("Sorted processing Times :");  
    for (int i = 0; i < n; i++)  
        System.out.print (arr[i] + " ");
```

```
    System.out.println();
```

```
    double median;  
    if (n % 2 == 0)  
        median = (arr[n/2 - 1] + arr[n/2]) / 2.0;  
    else  
        median = arr[n/2];
```

```

int count = 0;
for (int i = 0; i < n; i++) {
    if (arr[i] > median)
        count++;
}
int difference = arr[n-1] - arr[0];

System.out.println("Median: " + median);
System.out.println("Orders Above Median: " +
                    count);
System.out.println("Difference: " + difference);

sc.close();
}
}

```

Input: 7

12 25 18 30 15 22 40


```
PS C:\Users\Dell\Downloads> cd "c:\Users\Dell\Downloads\" ; if ($?) { javac MergeSortMedium.java } ; if ($?) { java MergeSortMedium }
7
12 25 18 30 15 22 40
Sorted Processing Times:
12 15 18 22 25 30 40
Median: 22.0
Orders Above Median: 3
Difference: 28
```

3. QUICK SORT (EASY):-

Problem:- Sort the given server response times in ascending order using Quick Sort Algo. Built-in sorting methods not allowed.

Constraints: $1 \leq N \leq 100000$

$1 \leq \text{response times} \leq 10000$.

Algo:- 1). Read N.

2. Read the response time into array.
3. Select last element as Pivot.
4. Partition the array such that elements smaller than the pivot are placed to its left and larger elements to its right.
5. Recursively apply Quick sort to the left and right.
6. Print the sorted array.

Java Code:- `import java.util.Scanner s;
public class QuickSortEasy {`

`static int partition(int arr[], int low, int high)
{
 int pivot = arr[high];
 int i = low - 1;`

`for (int j = low; j < high; j++) {`

`if (arr[j] < pivot) {
 i++;`

```

    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
}

int temp = arr[i+1];
arr[i+1] = arr[high];
arr[high] = temp;

return i+1;
}

```

```

static void quickSort (int arr[], int low,
                        int high) {
    if (low < high) {
        int pi = partition (arr, low, high);
        quickSort (arr, low, pi-1);
        quickSort (arr, pi+1, high);
    }
}

```

```

}

public static void main (String [] args)

```

```

{
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();
    quickSort (arr, 0, n-1);
}

```

```

    for (int i = 0; i < n; i++)

```

```

        System.out.print (arr[i] + " ");
    }
    sc.close();
}

```

Input : 6
120 45 78 200 60 95

```
DIFFERENCE 20
● PS C:\Users\Dell\Downloads> cd "c:\Users\Dell\Downloads\" ; if ($?) { javac QuickSortEasy.java } ; if ($?) { java QuickSortEasy }
6
120 45 78 200 60 95
45 60 78 95 120 200
○ PS C:\Users\Dell\Downloads> S
```


4. Quick Sort (MEDIUM):-

Problem: Sort the given trade values in descending order using the Quick Sort algo. After sorting display top 5 highest trade values, calculate their average, count how many trade values are greater than the overall average of all trade values.

Constraints: $5 \leq N \leq 100000$
 $1 \leq \text{Trade value} \leq 10^9$

- Algo:
1. Read element N.
 2. Read the trade values into an array.
 3. Calculate the overall average of all trade values.
 4. Sort the array in descending order using Quick Sort.
 5. Display the sorted array.
 6. Print the Top 5 highest trade values.
 7. Calculate and display the average of the Top 5 values.
 8. Count and display the no. of trade values greater than the overall.

Java Code: `import java.util.Scanner;`
`public class QuickSortMedium{`

```

static int partition (long arr[], int low,
                    int high) {
    long pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] > pivot) {
            i++;
            long temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    long temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;
    return i+1;
}

```

```

static void quicksort (long arr[], int low,
                    int high) {
    if (low < high) {
        int pi = partition (arr, low, high);
        quicksort (arr, low, pi - 1);
        quicksort (arr, pi + 1, high);
    }
}

```

```

public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt();
    long arr[] = new long[n];
    long sum = 0;
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextLong();
        sum += arr[i];
    }
}

```

```
double overallAverage = (double)sum/n;
quickSort (arr, 0, n-1);
```

```
System.out.println("Sorted array values:");
for(int i=0; i<n; i++)
```

```
System.out.print (arr[i] + " ");
System.out.println();
System.out.print ("Top 5: ");
```

```
long sum = 0;
for(int i=0; i<5; i++) {
    System.out.print(arr[i] + " ");
    topSum += arr[i];
}
```

```
double topAverage = (double) topSum/5;
int count = 0;
for(int i=0; i<n; i++) {
    if (arr[i] > overallAverage)
        count++;
}
```

```
System.out.println();
System.out.print ("Average of top 5: " +
    topAverage);
System.out.println ("Values Above overall
    Average: " + count);
```

```
sc.close();
```

```
}
```

Input: 8

500 1200 700 2000 300 1500 3000

```
PS C:\Users\DeII\Downloads> cd "c:\Users\DeII\Downloads\" ; if ($?) { javac QuickSortMedium.java } ; if ($?) { java QuickSortMedium }
8
500 1200 700 2000 900 1500 3000 2500
Sorted Trade Values:
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040.0
Values Above Overall Average: 3
```


5. HEAPSORT (EASY):-

Problem: Sort the given scores in ascending using Heap sort algo.

Constraints: $1 \leq N \leq 100000$
 $0 \leq \text{Score} \leq 1000000$

Algo: 1) Read N.
2. Read scores into array.
3. Build a max Heap from array.
4. Swap the root with the last.
5. Reduce the Heap size by one.
6. Heapify the root again.
7. Repeat 4-6 until sorted.
8. Print.

```
Java code: import java.util.Scanner;
public class HeapsortEasy {
    static void Heapify (int arr[], int n,
                        int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;
        if (right < n && arr[right] > arr[largest])
            largest = right;
        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heapify (arr, n, largest);
        }
    }
}
```

```

static void HeapSort (int arr[]) {
    int n = arr.length;
    for (int i = n/2 - 1; i >= 0; i--)
        heapify (arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify (arr, i, 0);
    }
}

public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();
    heapSort (arr);
    for (int i = 0; i < n; i++)
        System.out.print (arr[i] + " ");
    sc.close();
}

```

Input : 6

850 1200 350 600 1500 1000


```
PS C:\Users\Dell\Downloads> cd "c:\Users\Dell\Downloads\" ; if ($?) { javac HeapSortEasy.java } ; if ($?) { java HeapSortEasy }  
6  
850 1200 950 600 1500 1000  
600 850 950 1000 1200 1500
```

6. Heap sort (Medians):-

Problem: sort the given emergency response times in ascending order using the heap sort algo. After sorting find the fastest response time, slowest response time, average response time, no. of cases handled faster than the average and the percentage.

Constraints: $1 \leq N \leq 100000$
 $1 \leq \text{Time} \leq 500$

Algo:

1. Read the no. of N .
2. Read Response time.
3. Calculate the response sum.
4. Build a max heap.
5. Perform Heapsort in ascending.
6. Display.
7. Find the fastest.
8. Find the slowest.
9. Calculate average response.
10. Count the no. of response time less than the average.
11. Calculate the percentage of cases.
12. Display.

```

Java code: import java.util.Scanner;

public class HeapSortMedium {

    static void heapify (int arr[], int n, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify (arr, n, largest);
        }
    }
}

```

```

static void heapSort (int arr[]) {
    int n = arr.length;
}

```

```

for (int i = n - 1; i > 0; i--) {
    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;
}

```

```
    } heapify(arr, i, 0);  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);
```

```
    int n = sc.nextInt();  
    int arr[] = new int[n];
```

```
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        arr[i] = sc.nextInt();  
        sum += arr[i];
```

```
    }  
    heapSort(arr);
```

```
    System.out.println("Sorted Response Time");
```

```
    for (int i = 0; i < n; i++)  
        System.out.print(arr[i] + " ");
```

```
    System.out.print(arr[i] + " ");
```

```
    System.out.println();  
    int fastest = arr[0];  
    int slower = arr[n-1];
```

```
    double average = (double) sum / n;
```

```
    int count = 0;  
    for (int i = 0; i < n; i++) {  
        if (arr[i] < average)  
            count++;
```

```
    }
```

```

double percentage = (double) count / n * 100;
System.out.println("Fastest: " + Fastest);
System.out.println("Slowest: " + Slowest);
System.out.printf("Average: %.2f %n", average);
System.out.println("Cases faster than Average"
                  + count);
System.out.printf("Percentage: %.2f %n",
                  percentage);

```

```

    sc.close();
}
}

```

Input : 8
12 7 15 9 20 5 10 8

```
PS C:\Users\Dell\Downloads> cd "c:\Users\Dell\Downloads\" ; if ($?) { javac HeapSortMedium.java } ; if ($?) { java HeapSortMedium }
8
12 7 15 9 20 5 10 8
Sorted Response Times:
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
```